

Using Python with VSCode

In this module, we will learn how to set up VSCode to work with the Python programming language, and how to create a Python project workspace. We will also look at basic Python syntax. Before we start, let's make sure we understand the needed tools and what they do:

Python: The python interpreter that runs your Python code.

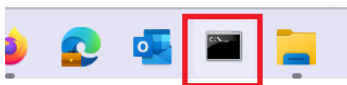
VSCode: The development environment application where you write and edit code and organize your code files.

Python extension: adds Python language support to VSCode. Think of it as a software component that makes Python work well inside VSCode.

Pre-requisites

You should have both **VSCode** and **Python** installed on your computer. Let's check by running a few commands to verify that it is the case:

- Open the command-prompt window:



- Check if VSCode is installed by typing the command `code --version` followed by pressing the Enter key.

If you see a version number (e.g., 1.117.0, as shown below), then VSCode is installed. **NOTE: Your VSCode version can be different than mine and that's OK.**

```
C:\Users\alfred.nehme>code --version
1.117.0
```

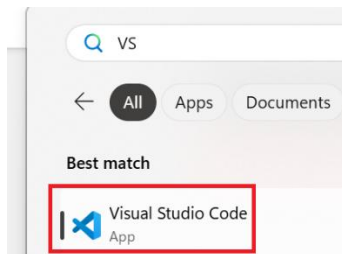
- Check if Python is installed by typing the command `python --version` followed by pressing the Enter key. You should see the word Python followed by some version number (e.g., 3.13.1). **NOTE: Your Python version can be different than mine and that's OK.**

```
C:\Users\alfred.nehme>python --version
Python 3.13.1
```

If both VSCode and Python are installed, you can move to the next section: Installing the Python Extension.

Installing the Python Extension

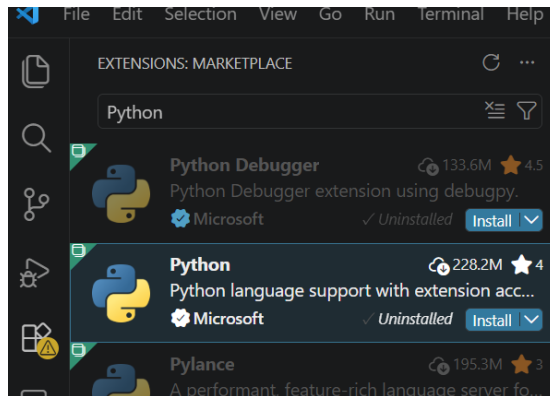
- Open VSCode by clicking the Windows button on the keyboard and start typing VSCode. The VSCode menu comes into view. Click it to open VSCode.



- Click menu **View → Extensions**.

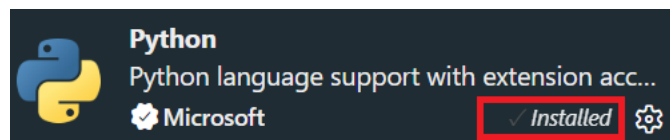
This opens the EXTENSIONS pane. From here you can search for and install freely available extensions to make VSCode work with the programming language that you want. In our case we want it to work with Python.

- a. In the search box, type the word Python.
- b. From the extensions that appear click the one shown below (authored by Microsoft):



- c. Click the **Install** blue button.
(Wait a few seconds for installation to complete).

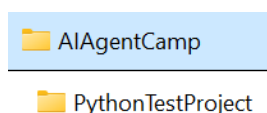
When installation is complete, the Install blue button disappears, and you should see the label “Installed”.



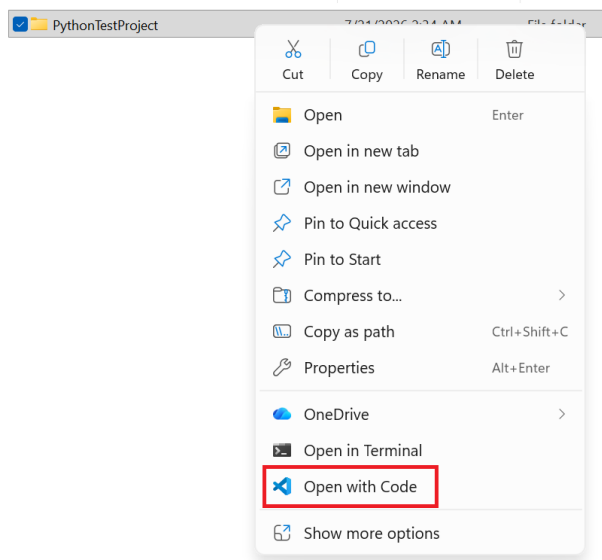
Creating a Workspace folder and a virtual environment

Create a workspace folder

1. On your PC create a folder named “AIAgentCamp”. We will put all our code in subfolders under this folder.
2. Under AIAgentCamp, create a subfolder named “PythonTestProject”:



3. Right-click folder PythonTestProject and choose menu **Open with Code**.
(VSCode will likely ask you if you trust the authors of this folder. Answer **Yes, I trust the authors**).

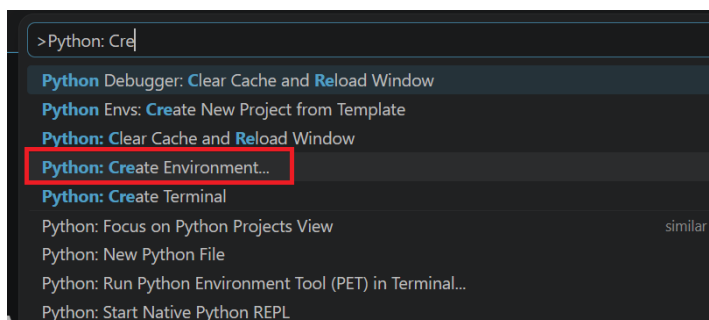


By starting VSCode in a folder, this folder becomes your **workspace**.

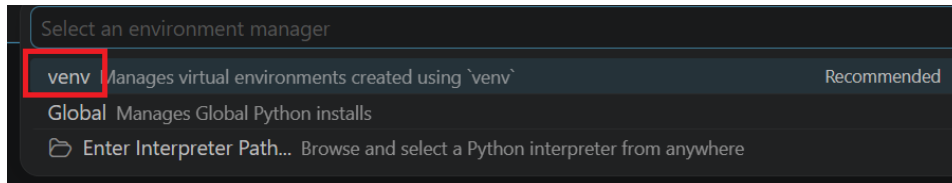
Create a virtual environment

The best practice among Python developers is to use a project-specific **virtual environment**. Once you activate that environment, any packages you then install are isolated from other environments, reducing many complications that can arise from conflicting package versions.

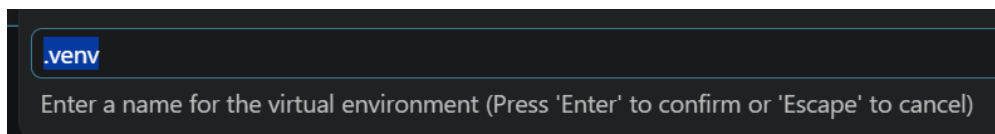
4. In VSCode, click menu **View** → **Command Palette...**
5. Start typing **Python: Create Environment**. This brings the menu into view.



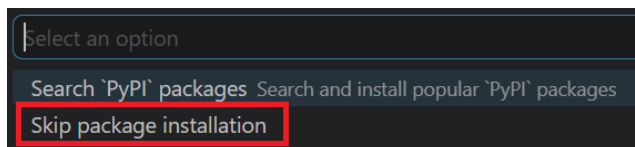
6. Click the **Python: Create Environment...** menu. This presents a list of environment types.
7. Click the **venv** environment type.



8. The venv command then presents a list of python versions that can be used for your project. Select the version of python that is installed on your PC.
9. Then just press Enter when you see this presented:

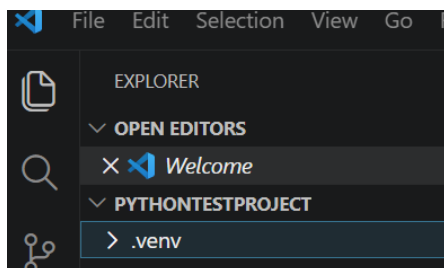


And choose menu Skip package installation.



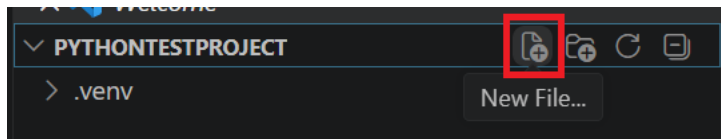
At the lower right of VSCode, you will see progress dialogs telling you that VSCode is creating the environment. (Wait for it to finish).

10. When done you should see a **.venv** folder created under PythonTestProject:

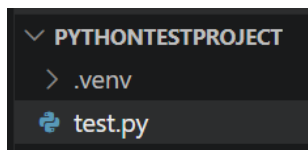


Test that we can write python code and run it

11. While PYTHONTESTPROJECT is selected, click the **New File...** toolbar button, and create a file named `test.py`



You should now have `test.py` under PYTHONTESTPROJECT.

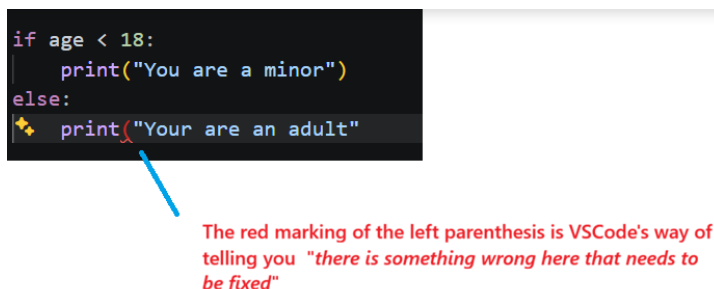


12. In `test.py`, enter code that looks like this:

```
age = 16

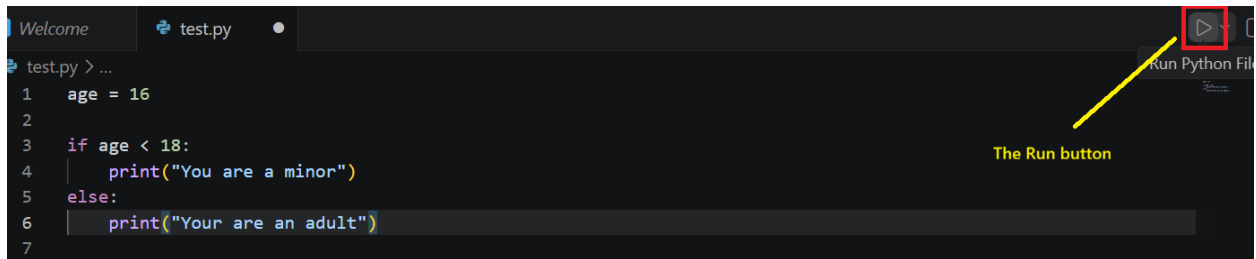
if age < 18:
    print("You are a minor")
else:
    print("Your are an adult")
```

Note that if you make a mistake (e.g., you forget to close the parenthesis of the print function), VSCode warns you by marking the statement where the mistake is as shown in the example below:



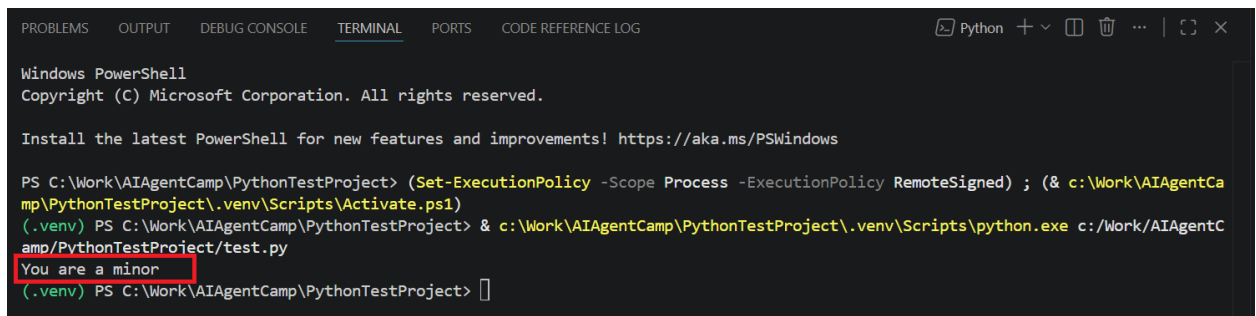
So, when you type code and you see markings like this, it is a hint for you that something is not quite correct that you need to fix.

13. Now that we have written some code, we need to run it to test that it works. We do this using the **Run Python File** button at the far right of the `test.py` file tab.



Click the **Run Python File** button.

If all is correct, you should see the output of your program in the TERMINAL tab in VSCode. In this case we see the correct message getting printed: "You are a minor"



14. Change the value of the `age` variable to a number bigger than 18 and run the code again. You should now see that the other message got printed: "You are an adult".
15. In many cases, it is efficient to organize snippets of code into functions. Doing so improves clarity and re-usability. In Python we write functions using the **def** keyword. Let's practice writing functions in Python:

- Change the code in `test.py` to look like the below (be careful not to make syntax errors).

```

def pounds_to_kilograms(m):
    # 1 pound = 0.4535 Kg
    kg = m * 0.4535
    return kg

def kilograms_to_pounds(m):
    pounds = m / 0.4535
    return pounds

age = 34
m1 = 4.45      # Some mass in pounds
m2 = 2.3       # Some mass in kilograms
m3 = 64.13     # Some other mass in kilograms

kilograms = pounds_to_kilograms(m1)
print(str(m1) + " lbs = " + str(kilograms) + " kg")

pounds = kilograms_to_pounds(m2)
print(str(m2) + " kg = " + str(pounds) + " lbs")

pounds = kilograms_to_pounds(m3)
print(str(m3) + " kg = " + str(pounds) + " lbs")

if age < 18:
    print("You are a minor")
else:
    print("Your are an adult")

```

Notice how we used the `def` keyword to define two functions: `pounds_to_kilograms` and `kilograms_to_pounds`.

The basic structure of a Python function looks like this:

```

def function_name(arguments):
    # function body
    return value

```

The `def`, `:`, and indentation are required. Unlike other programming languages, indentation is required in python, and writing a function like the one shown below is incorrect:


```
def function_name(arguments):  
    # function body  
    return value
```



Incorrect function syntax due to absence
of indentation

Although our two functions take one argument (m), you can write functions that take more than one argument. Simply separate arguments by commas. For example, below is a function that takes three arguments: age, grade, school_name:

```
def some_function_name(age, grade, school_name):  
    # function body
```

- b. Save the changes you made by clicking menu **File → Save**. Then run the code by clicking the **Run Python File** button.
- c. Observe the output in the TERMINAL pane. Does the output is what you expected from your code?
- d. Because you live in the USA, you probably know your weight in pounds. To find out your equivalent weight in kilograms, change the value of m1 to your weight in pounds and re-run the code (by clicking the **Run Python File** button).

In the code above we named the functions pounds_to_kilograms and kilograms_to_pounds. The convention in python is to use words in lowercase characters and separate the words by underscore. This is not a rule, but a convention. In programming, it is always recommended to follow conventions. So, we stick with this rule and follow it

Incorrect name: does not follow convention

```
def calculateGrade():  
    ...  
    ...
```

Incorrect name: does not follow convention

```
def CalculateGrade():  
    ...  
    ...
```

Although it follows the naming convention, the name cal_g is not descriptive to exactly know what this function does. So, avoid such simplifications.

```
def cal_g():  
    ...  
    ...
```

Correct name: follows convention of all lowercase letters and words separated by # underscore. The name is also descriptive (i.e., it is clear what this function does)

```
def calculate_grade():  
    ...  
    ...
```

Note that this naming convention applies to python only. Other programming languages (e.g., Java) can have different naming conventions.

This naming convention for functions also applies to variable names in python. For example:

glucoseLevel	# Incorrect: does not follow convention
GlucoseLevel	# Incorrect: does not follow convention
glucose_level	# Correct: follows convention and is descriptive enough
speed	# Correct: follows convention and is descriptive enough
avg_sc	# Incorrect: follows convention but not descriptive enough
total_price	# Correct: follows convention and is descriptive enough
4wd	# Illegal name: variable name cannot start with a number
account balance	# Illegal name: variable name cannot have empty space

For variables that are constant (i.e., their values are usually fixed and do not change as the program runs), programmers like to use all upper-case characters for such variables. This is common and is usually considered OK.

```
PI = 3.14           # OK (value of pi does not change)
MONTHS_IN_YEAR = 12 # OK (number of months in a year does not change)
SALES_TAX = 0.1     # Incorrect: tax can be different in different cities/states
```

Using a while loop to repeat steps in a program

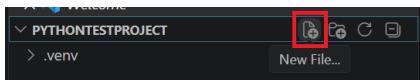
In many cases, a program may want to do the same set of operations repeatedly. For example, assume you want to write a program that converts temperature in degrees Celsius to degrees Fahrenheit. We don't want our program to just do it once and terminate. Instead, we want to allow the user to make as many conversions as she wants until she decides to quit the program (by typing some word such as quit or exit). To do that we use a while loop. The syntax for the while loop looks like this:

```
while(expression):
    # code
    # more code...
```

where expression is any code that evaluates to a Boolean (True or False). As long as expression evaluates to a True value, the code of the while construct executes repeatedly. To exit the while loop at any point, we use the **break** keyword. In some cases, we want to skip the remaining code in the while loop and start at the top of while again. To do that, you use the python **continue** keyword.

We will do an example that demonstrates the use of while(), continue, and break.

16. While PYTHONTESTPROJECT is selected, click the **New File...** toolbar button, and create a file named temp_converter.py



17. Enter the following code in temp_converter.py:

```

# stdout gives us more granular control than just the print() function
from sys import stdout      # import stdout from the sys package

# We want to impress our users and make our program look professional.
# At the same time, we want it to look pretty and fun. So we use some colors for text
BLUE = "\033[34m"          # Questions will be printed in blue
RED = "\033[31m"           # Errors will be printed in red (to draw the attention of the user)
DEFAULT_COLOR = "\033[0m"  # Reset to default color

# define the convert_to_fahrenheit function
def convert_to_fahrenheit(celsius_temperature):
    # The formula to convert a temperature in °C to °F is: Temp_F = 1.8 x Temp_C + 32
    fahrenheit_temperature = 1.8 * celsius_temperature + 32
    return fahrenheit_temperature

# Helper function to check if a string s represents an integer or a float
def is_a_number(s):
    try:
        float(s) # Attempt to convert the string to a float
        return True
    except ValueError:
        return False

```

```

# We want our program to repeat some set of operations indefinitely until the user tells it to stop
# To do that we use a combination of while, break, and continue.
while(True):
    stdout.write(BLUE + "Enter a temperature in oC (or exit to quit): " + DEFAULT_COLOR)
    entry = input()    # The input function returns a string

    # Check if the user pressed Enter without typing anything. We use the len() function which
    # returns the length of a string. If the length is 0, we know that they didn't type anything
    if len(entry) == 0:
        continue    # Skips the code below and repeats the while from the start

    if entry == "exit":
        # User decided to quit. We use break to exit the while loop
        break

    # Notice the difference between continue and break.
    # continue: Instructs the program to immediately jump to the first statement after while():
    # break: Immediately exits the while loop (in this case the program continues at code that is
    # immediately after the end of the while loop)

    # Users can be careless and make mistakes by, for example, entering something other than a number
    # Check if what was entered represents a number
    if not is_a_number(entry):
        stdout.write(RED + "Invalid temperature value. Please enter a valid temperature value\n")
        continue    # Skips the code below and repeats the while from the start

    # If we get to here, it means the user did not press Enter without typing anything,
    # did not type exit, and did type a string that is a valid number.
    # We are now confident to use our convert_to_fahrenheit function to perform the calculation
    temperature = float(entry)    # Converts entry from a string type to a number type

    result = convert_to_fahrenheit(temperature)
    stdout.write(DEFAULT_COLOR + entry + " oC = " + str(result) + " oF\n")

    # After printing the result, the code inside the while will repeat again at the first statement
    # after while():, which is what we want.

```

18. Save the changes to file `temp_converter.py` by choosing menu **File** → **Save**.

19. Run the code in file `temp_converter.py` by clicking the **Run Python File** button.

Using and importing python packages

It wouldn't be efficient if we must write from scratch every function that our code needs, especially if the function does something generic that can be used in different code and programs. Fortunately, software developers write many useful functions and group related ones into **packages**. For example, there is a python package called **fs** that contains many useful functions related to doing things in the file system like checking if a file exists,

creating a new file, etc. Another important package is called **os** that contains functions related to doing things in the operating system, such as getting the username of the logged-in user.

Therefore, think of a python **package** as a group of useful functions specific to some domain. (NOTE: in other programming languages packages are also known as **libraries**).

Python packages are of two types: **built-in** (come with python) and **external**, which need to be installed separately. Built-in packages can be directly imported using the **import** keyword without any installation. This allows access to all the functions and variables defined in the package. For example, assume you want to calculate the volume of a sphere, which is defined as

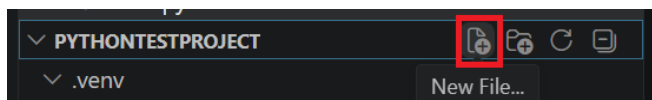
$$V_{\text{sphere}} = \frac{4}{3} \pi r^3$$

Where π is the pi constant and r is the sphere radius. Given the value of r, to perform this calculation we need the value of π and to be able to raise r to the power of 3 (r^3). The python **math** package allows us to easily do this. To import a package in python, we use the **import** keyword.

```
import <package name>
```

*Importing and Using a **Built-In** Package*

20. In VSCode, make sure PYTHONTESTPROJECT is selected, then click the **New File...** toolbar button to create a new file named test2.py.



21. In `test2.py`, enter this code:

```
# We use the import keyword to import a python package

import math      # import the math package
import sys       # import the sys package

BLUE = "\033[34m" # We print questions in blue
DEFAULT_COLOR = "\033[0m" # Reset to default color

def sphere_volume(r):
    volume = math.pi * math.pow(r, 3)
    return volume

sys.stdout.write(BLUE + "Enter a sphere radius: " + DEFAULT_COLOR)
radius = input() # The input function returns a string

# We must convert the string radius to a number using the float() function.
volume = sphere_volume(float(radius))

sys.stdout.write(DEFAULT_COLOR + "The volume of the sphere with radius " + radius + " is: " + str(volume))
```

To use a package in code, we just use the `import <package name>` syntax, as was done in the first two statements:

```
import math
import sys
```

Because both `math` and `sys` are python built-in packages, we didn't have to install them to use them. If we want to use a package that is not built-in, we need to first install it, and then import it using the `import` keyword (an example of how to do that is given later in this module)

22. Click menu **File** → **Save** to save the changes you made to `test2.py`.

Then, click the **Run Python File** button to run the code of `test2.py`.

23. In the **TERMINAL** pane, enter a radius at the blue prompt, then press the Enter keyboard button. You should see the output of the program with the sphere volume calculated.

```
Enter a sphere radius: 65.8
The volume of the sphere with radius 65.8 is: 895009.311258104
```

In the `sphere_volume` function, we used the `pi` constant defined in the `math` package (`math.pi`), and the `math.pow()` function to raise a number to any power. We also used the `sys.stdout.write()` function from the `sys` package to write to standard output. We could have used the `print()` function to print to standard output, but `sys.stdout.write()` gives us more granular control (e.g., printing in a specific color, not appending a newline character after each printed line, etc.).

*Importing and Using an **External** Package*

We often want to use functionalities from **external** packages. An external package is a package that is not built-in in python. It is available in the Python Package Index repository (pypi.org) where we can download it from and install it. Packages are usually domain-specific, and there are hundreds of thousands of them.

- Open your browser and go to <https://pypi.org/>
- In the Search textbox, enter the word **statistics** and press the Enter key. You will see lots of packages related to statistics and you can choose the one(s) that are suitable for your statistics calculations.
- In the Search textbox, enter the word **http** and press the Enter key. Again, you will see lots of packages related to http.

There is an external package for pretty much anything you can think of. Not only that, but developers all over the world continue to write new packages and add them to the repository. When you develop enough expertise in some domain, you too can write a package and add it to the repository.

- Go to [bing.com](https://www.bing.com) or [google.com](https://www.google.com) and search for this:

“how many external python packages exist?”

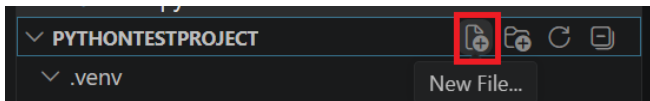
To download and install an external package we use a tool called **pip**, which is the standard package manager for python. You should have pip on your PC because it was automatically installed with python. You can verify that it is the case by running **pip --version** from the command prompt.

```
C:\Users\alfred.nehme>pip --version
pip 24.3.1 from C:\Python313\Lib\site-packages\pip (python 3.13)
```

Once we download and install an external package using pip, we then use the `import`

keyword as was done in the previous example with built-in packages such as `math` and `sys`.

24. In VSCode, make sure `PYTHONTESTPROJECT` is selected, then click the **New File...** toolbar button to create a new file named `test3.py`.



25. We will experiment with an external package named `matplotlib`, which is a package for creating static, animated, and interactive visualizations in python.

In the TERMINAL pane, run the command `pip install matplotlib` to download and install the `matplotlib` package:

```
(.venv) PS C:\Work\AIAgentCamp\PythonTestProject> pip install matplotlib
```

(Wait for `matplotlib` and other dependencies it depends on to finish download and install)

Once download is complete, if you expand the `.venv/Lib/site-packages` folder under `PYTHONTESTPROJECT` in VSCode, you should see the `matplotlib` package and also many new packages that `matplotlib` depends on.

26. Enter this code in `test3.py`

```
import matplotlib.pyplot as plt

# The [..., ..., ..., etc.] in python allows us to define a list. Unlike a regular
# variable that can hold a single value, a list can hold many values.

# Let's define some X and Y values
x = [1, 2, 3, 4] # Assume these numbers represent Temperature
y = [2, 4, 6, 8] # Assume these numbers represent Pressure

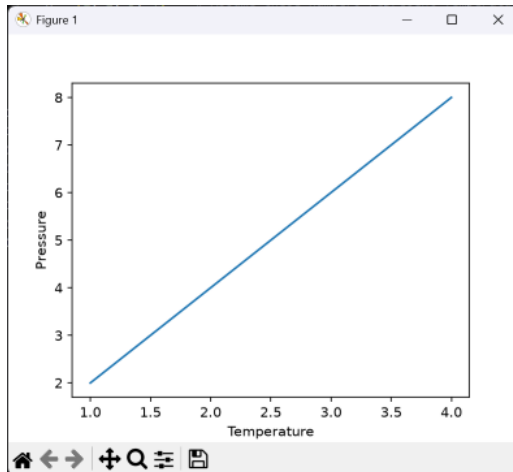
# It is always important to label the axes of a plot (so that readers know what it represents)
plt.xlabel('Temperature')
plt.ylabel('Pressure')

# Create the plot
plt.plot(x, y)

# Show the plot
plt.show()
```

27. Click menu **File** → **Save** to save the changes you made to `test3.py`.
Then, click the **Run Python File** button to run the code of `test3.py`.

You should see the plot getting created in a separate window



In the first statement we used

```
import matplotlib.pyplot as plt
```

The `as` keyword allows you to give the package an alias. In this case we used `plt` as the alias for `matplotlib.pyplot`. It is just simpler to type `plt` than to type `matplotlib.pyplot`.

In all examples above, we used the following syntax to import a module:

```
import <package name>
```

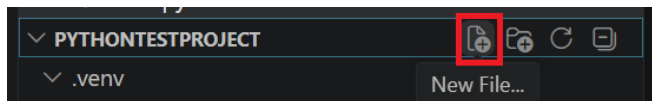
where `<package name>` is the name of the package to import. For example, we imported the `math` package using `import math`. In this case you can use all the functions, classes, and constants defined in `math`, such as `math.pi` or `math.sqrt()` to calculate the square root of a number, etc.

In some cases, you might be interested in only a specific functionality in a package. Python supports an alternative way to import these functionalities using a syntax like this:

```
from math import pi, sqrt
```

If you import in this way, you can then use the `pi` constant and the `sqrt()` function without the `math.` prefix. Let's do a quick test to make sure you understand this syntax:

28. In VSCode, make sure `PYTHONTESTPROJECT` is selected, then click the **New File...** toolbar button to create a new file named `test4.py`.



29. Copy all the code from file `test2.py` and paste it into `test4.py`.

30. Update the code to make it look like this:

```
from math import pow      # import the pow function from the math package
from math import pi      # import the pi constant from the math package
from sys import stdout    # import stdout from the sys package

BLUE = "\033[34m"
DEFAULT_COLOR = "\033[0m" # Reset to default color

def sphere_volume(r):
    volume = pi * pow(r, 3) # We can use pi and pow without writing math.
    return volume

# Here we can use stdout without using the prefix sys. (as we did in test2.py)
stdout.write(BLUE + "Enter a sphere radius: " + DEFAULT_COLOR)
radius = input() # The input function returns a string

# We must convert the string radius to a number using the float() function.
volume = sphere_volume(float(radius))

stdout.write(DEFAULT_COLOR + "The volume of the sphere with radius " + radius + " is: " + str(volume))
```

31. Save the changes you made to `test4.py` by clicking menu **File** → **Save All**.

Notice how we changed the way we imported `pow`, `pi`, and `stdout`. We used the syntax `from <package name> import ...`

And once we did that, we were able to access `pi`, `pow`, and `stdout` without needing to use the `packagename.` syntax.

32. Click the **Run Python File** button to run the code of `test4.py`.

(The program output should be like `test2.py` because we didn't change the logic of the program; we just changed how we import the functionalities we want from packages `math` and `sys`.)

Exercise to do on your own to practice what we learned above

In this exercise, you will create a brand-new python project in VSCode, use an external package named `pandas` to read the content of a comma-delimited file, and find the average score of a few columns from this file. I will give you a few hints of how to get started, but you need to do the work on your own to practice what we learned.

- To create a new VSCode project, follow steps 2 to 10 starting on page 3 of this document. Give your project folder the name **AnalyzingData**.
- In the project, create two python files: **analyzer.py** and **data.csv**.
- In **data.csv**, add 10 rows of random data following the format shown below (comma-delimited). We are assuming that this data is used to save quizzes/tests scores for 10 students. The first row is the columns' header shown below in blue. You should then enter 10 rows of random grades for 10 students (students with IDs 10001 to 10010). You can make up your own numbers.

student_id,quiz1,quiz2,quiz3,midterm,final

10001,82.5,87.3,78.8,94.5,89.6

10002,77.6,68.4,83.5,75.0,83.8

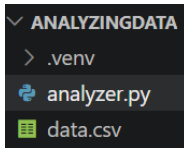
...

...

10010,88.7,92.0,94.3,86.5,89.4

(Remember that data items must be separated by commas (,))

When done, your **data.csv** file should have a total of 11 rows: the columns header row + grades of 10 students. And your project folder structure should look like this:



Save the changes you made by clicking menu **File** → **Save all**.

- The pandas' package is an external package (not built-in into python). Remember that we had to take an extra step before we could use external packages (see step 25 on page 17 for an example). In this case the name of the package is **pandas**.

NOTE:

Whenever you use a package for the first time, you always need to look at its documentation to find out what it can do, functions it supports, example code you can look at, etc. The documentation of the pandas' package can be found [here](#). You can also see a few tutorials by clicking the [Getting started](#) link at the top of the page, then expand the [Getting started tutorials](#) from the left of the page. The [How do I read and write tabular data?](#) tutorial is something that you can look at to learn how to read your data.csv file.

- To open the grades.csv file, use code like this:

```
import pandas as pd

grades = pd.read_csv("data.csv", header=None, skiprows=1)
```

The `read_csv` function opens the file, reads the data, and saves it in variable `grades`, an object of type `DataFrame`. You can think of a `DataFrame` object as a 2-dimensional tabular data that you can manipulate in your code.

If you are interested in checking the type of a variable (e.g., the `grades` variable), you can use code like this:

```
print(type(grades))
```

The `type` function returns the type of the variable passed to it. If you try this code with the `grades` variable, you get back a `DataFrame`

We plan to do calculations on the columns (e.g., finding the average of the midterm exam) and we are not interested in the first row of the data (the header) because the header is text and we can't do numeric calculations on text. To skip reading the header, we use the parameter `skiprows=1`, which makes the `read_csv` function ignore the first row when it reads the file data. In this case, pandas automatically assigns an index to each column (0 to 5 in this case because there are 6 columns), and we can use these indexes to refer to a particular column. You can check what grades now have by printing it and running the program:

```
import pandas as pd

grades = pd.read_csv("data.csv", header=None, skiprows=1)
print(grades)
```

- The following code shows how to access data of a specific column (pandas assigns to columns an integer number – the column index – starting with 0). The code below shows how to access data of a specific column:

```
import pandas as pd

grades = pd.read_csv("data.csv", header=None, skiprows=1)

# To access the data of a specific column we use its index.
# For example, student_id is in column 0, and we can get the
# student_id columns data like this:
students_ids = grades[0]
print(students_ids)

# And the midterm grades are in column 4
midterm_grades = grades[4]
print(midterm_grades)
```

TODO: Complete the program to output the following:

Average (also known as mean) score on Quiz1

Average score on Quiz2

Average score on Quiz3

Average score on Midterm

Average score on Final

Midterm lowest score

Midterm highest score

Final lowest score

Final highest score

List of student IDs and their final exam grade who scored below the average on the final

For example, below is the output from my data:

```
Quiz1 average score:    79.28
Quiz2 average score:    81.38
Quiz3 average score:    84.40
Midterm average score:  84.38
Final average score:    84.38

Midterm exam:
-----
Lowest:  75.0
Highest: 94.5

Final exam:
-----
Lowest:  79.5
Highest: 95.5

Students who scored below the average on the final:
-----
10002 (83.8)
10004 (81.5)
10007 (83.5)
10008 (79.5)
```

NOTE: Not just correct results, but clarity of results is important in programming. It makes it simpler for people to read and understand what your program is returning.

So, make sure your output is clean and well aligned, clear, and correctly labeled as shown above. Also adding an empty line between one section of your output and another is recommended as it separates the results in sections that are visually distinguishable.

Note that you need to use functions that exist in pandas to find the minimum, maximum, mean, etc. After all, that's why we are using this package; to use its useful functionalities instead of us having to calculate them.

A good place to start is to look at the pandas' package documentation and search the web. For example:

"Find average value of pandas dataframe column"

"How to get the minimum value in a pandas dataframe column"

You will get different suggestions. In programming, there are usually always different ways of doing the same thing. So, you might find different pages suggesting how to calculate the mean. Just choose one.